

Generics

Generics

Come usare tipi parametrizzati per collezioni e classi riutilizzabili.

Il problema che risolvono

Hai già visto codice come questo:

```
ArrayList nomi = new ArrayList<>();
```

La parte `String` tra parentesi angolari dice che la lista contiene stringhe.

Questa idea si chiama **generics**.

Tipi più sicuri

Senza generics, una lista potrebbe contenere di tutto: testi, numeri, oggetti diversi.

Con i generics, Java controlla il tipo.

```
ArrayList nomi = new ArrayList<>();  
  
nomi.add("Luca");  
nomi.add("Sara");  
// nomi.add(10); // errore
```

`10` è un `int`, non una `String`. Java lo segnala subito.

Generics nelle collezioni

Esempi comuni:

```
ArrayList nomi = new ArrayList<>();
ArrayList numeri = new ArrayList<>();
HashMap prezzi = new HashMap<>();
HashSet parole = new HashSet<>();
```

`Integer` è la versione oggetto di `int`. Nelle collezioni Java si usano tipi oggetto, non primitivi.

Altri esempi:

- `Double` per `double`
- `Boolean` per `boolean`
- `Character` per `char`

Creare una classe generica

Puoi creare anche le tue classi generiche.

```
public class Scatola {
    private T valore;

    public Scatola(T valore) {
        this.valore = valore;
    }

    public T getValore() {
        return valore;
    }
}
```

`T` è un parametro di tipo. Significa: il tipo verrà scelto quando userai la classe.

Usare la classe generica

```
public class Esempio {  
    public static void main(String[] args) {  
        Scatola scatolaNome = new Scatola<>("Luca");  
        Scatola scatolaNumero = new Scatola<>(42);  
  
        System.out.println(scatolaNome.getValore());  
        System.out.println(scatolaNumero.getValore());  
    }  
}
```

Output:

```
Luca  
42
```

La stessa classe `Scatola` funziona con tipi diversi, ma resta controllata.

Perche non usare Object

Potresti pensare di usare `Object`, il tipo piu generale.

```
public class Scatola {  
    private Object valore;  
}
```

Il problema e che poi perdi informazioni sul tipo e devi fare cast manuali.

I generics evitano molti cast e molti errori.

Regola pratica

Usa i generics quando una classe o una collezione deve lavorare con tipi diversi, ma vuoi mantenere controlli chiari.

Nella maggior parte dei programmi iniziali li incontrerai soprattutto con collezioni come `ArrayList` , `HashMap` e `HashSet` .